Surface Effects: a Language Extension for Deterministic Parallelism

Vitor Rodrigues* Matthew Fluet†

Abstract

This paper presents a region-polymorphic simply-typed λ-calculus with mutable references and deterministic implicit parallelism. Its distinctive feature is an executable effect-summary language: a computation is paired with a well-typed summary term that may depend on run-time values, is statically related to the computation, and is evaluated before the computation to obtain computed effects used by the parallel scheduling rule. We formalize static effects, dynamic traces, surface-effect terms, and evaluated computed effects. The mechanized development proves that verified summaries soundly approximate computation traces. Pairwise-disjoint and non-conflicting computed summaries imply deterministic structured traces, and the continuation-machine semantics proves finite-prefix type and trace safety for prefixes of potentially diverging executions, terminal soundness for completed runs, and the staged success/failure behavior of the dynamic parallel check. The calculus therefore separates static verification of executable summaries from a run-time scheduling decision. We do not assume a constant or bounded cost for summary evaluation: the precision and evaluation cost of a programmer-written summary remain design choices to be assessed independently of the soundness result.

Contents

1 Motivating Example **3**

2 Static, Dynamic and Surface Effects **4**

3 Syntax **4**

4 Operational Semantics **5**

5 Disjointness of Evaluated Surface Effects **7**

6 Static Semantics **8**

7 Correctness of Surface Effects **9**

8 Small-Step Runtime Safety **12**

9 Discussion and Related Work **14**

10 Conclusions **15**

A Deterministic Parallelism **16**

Introduction

The advent of multicore processors opened a brand new perspective on concurrent computing. However, the power of hardware parallelism implies increasing programming complexity and turns parallel

*Università degli Studi di Torino, Turin, Italy. vitor.rodrigues@di.unito.it
†Rochester Institute of Technology, New York, USA. mtf@cs.rit.edu

programs with shared state subject to unexpected behaviors that violate the principle of deterministic parallelism, such as deadlocks and data races. Hence, high-level programming languages are required to provide mechanisms for parallel computing and, at the same time, take advantage of the formal design that is already at use to prove sound properties about sequential programs.

The presented language is a region-polymorphic λ-calculus with mutable references, where typability is established using a type-and-effect discipline in the Lucassen–Gifford lineage (Lucassen and Gifford, 1988). Region effects and non-interference checks for deterministic parallelism are not, by themselves, the novelty claimed here; DPJ, for example, uses modular compile-time type and effect checking to guarantee deterministic semantics (Bocchino et al., 2009). The contribution of surface effects is narrower: a computation is paired with a statically verified, executable effect-summary term that can depend on run-time values and is evaluated before the computation to gate the parallel rule. This makes the summary an ordinary typed program term rather than only compiler metadata. The design is therefore related to inspector–executor approaches, which discover run-time dependence information before executing a conflict-free parallel schedule (Arenaz et al., 2004), but integrates the "inspector" into the source language and the metatheory through the correctness relation defined below.

Two distinguished summary terms delimit the approximation space implemented by the calculus. The term '`Top`' evaluates to the distinguished computed effect $\top$ and soundly approximates every dynamic trace. In the mechanized semantics, however, $\top$ conflicts with every computed effect and cannot satisfy the computed-effect disjointness premise; it therefore conservatively prevents the parallel rule from being applied. The term '`Pure`' evaluates to the empty computed-effect set and describes computations with no dynamic actions.

The central intuition is that one well-typed term is described by *another* well-typed term of the same language. A summary may inspect run-time values and, for example, compute a concrete region/location pair instead of retaining only a region-level read or write. Such a summary may distinguish accesses that a region-only static effect merges. The formal development does not define a global precision lattice relating static effects, computed effects, and traces, nor does it prove that a more precise summary is necessarily more expensive. The soundness claim is instead expressed by explicit approximation relations: static effects soundly cover dynamic actions, and a verified surface-effect term evaluates to a computed effect that soundly covers the trace of its paired computation.

A run-time decision is safe only through this proof chain. Correctness of the summary yields trace approximation; computed-effect disjointness and absence of computed conflicts are then transported to the dynamic traces; finally, the trace semantics establishes deterministic replay. Precision can therefore recover parallel executions that coarse region summaries cannot distinguish, but performance remains a separate question because the calculus contains no cost semantics or empirical evaluation.

Put simply, surface effects is a hybrid design solution for deterministic and implicit parallelism that balances the strengths and weaknesses of the existing solutions found in the literature (Bocchino et al., 2009; Flanagan et al., 2006; Kawaguchi et al., 2012; Kulkarni et al., 2007; Long et al., 2013; Lucassen and Gifford, 1988). Generically, approaches to implicit/explicit deterministic parallelism can be classified in the categories of "purely static" and "purely dynamic". The type of checking (static vs. dynamic) to be performed using the relation ($\otimes$) and the consequent main disadvantage (highly conservative vs. highly inefficient) is explained in Fig. 1. The middle-ground effect system based on surface effects is highlighted to emphasize that surface effects is a hybrid approach that relies both on an extended static type-and-effect system and on an extended set of syntax-directed evaluation rules.

	Check	**Main drawback**
Type-and-effect (static check)	$\epsilon_1 \otimes \epsilon_2$	may reject safe parallelism
Surface-effect pre-check	Θ_1, Θ_2	summary-evaluation cost
Dynamic access tracking	$\phi_1 \otimes \phi_2$	per-access tracking/validation

Figure 1: Middle-ground effect system: trade-off between pros and cons.

Let $(\!|\, e_1,\ e_2 \,|\!)$ be the syntax construct for implicit-parallel tuples of the surface language (see Section 3). The dynamic semantics is a continuation machine whose states contain a heap, an environment, a region map, an expression or returned value, and a continuation stack. A finite

execution emits a list of dynamic actions, which is converted into the structured traces used by the effect soundness theorems (see Section 4). The static semantics uses type-and-effect judgments in the form of $\Gamma \vdash e : \tau \,!\, \epsilon$, where Γ is a typing context, e is an expression, τ is a type and ϵ is a static effect (see Section 6). On the one hand, the static semantics of surface effects defines the back-triangle relation ($\blacktriangleleft$) to denote that the computational expression e is correctly described by the effect summary $\widehat{\Theta}$ (an expression of the surface effects mini-language). On the other hand, the effect summary is dynamically evaluated to a set of computed effects, Θ, at run time that soundly describe ($\sqsupseteq$) the trace produced by the execution of e.

Static region/effect systems such as DPJ make non-interference a modular compile-time obligation, but region-level summaries may merge concrete accesses that are distinct at run time (Bocchino et al., 2009). Runtime inspector–executor techniques address irregular dependences by discovering conflict-free work before executing it in parallel (Arenaz et al., 2004). Surface effects occupy a related middle position: the access summary is supplied as a typed source-language term, checked against the computation, and evaluated before the parallel computation. Unlike optimistic post-validation, the calculus does not start a parallel computation and then roll it back after a summary conflict; however, this fact alone does not establish that pre-checking is cheaper in total.

The formal contribution is therefore a sound scheduling interface rather than an asymptotic performance result. At compile time, an inductive correctness relation validates programmer-written computation/summary pairs. At run time, the summary terms evaluate to computed effects that may contain concrete region/location information. The mechanized development connects these stages: summary correctness gives trace approximation, summary-level safety gives a deterministic parallel trace, and deterministic trace replay gives equivalent final heaps. False positives remain possible because summaries are approximations and 'Top' is deliberately maximally conservative.

The contributions of this paper are:

- an executable effect-summary mini-language embedded in a region-polymorphic calculus with mutable references;
- an inductive correctness relation for statically verifying a computation against a programmer-written, value-dependent summary term;
- an operational semantics in which evaluated summaries gate an implicit parallel tuple using both computed-effect disjointness and conflict absence;
- a mechanized approximation theorem showing that a verified summary soundly covers the dynamic trace of its computation;
- a mechanized scheduler-soundness chain from safe computed summaries to deterministic structured traces and heap uniqueness under trace replay;
- a mechanized continuation-machine safety layer showing that every finite prefix of a well-typed execution remains type safe, trace typed, and not stuck, plus explicit terminal theorems for the checked and fallback outcomes of the dynamic parallel check.

The rest of this paper is organized as follows. We start by presenting a motivating example in Section 1. Section 2 gives the definitions of three sorts of effects: static, dynamic and surface. The syntax for types and the surface language, including the mini-language of surface effects, is given in Section 3. The static semantics of the type-and-effect system is presented in Section 6 and the operational semantics of the implicit parallel language is presented in Section 4. The disjointness of evaluated surface effects is discussed in Section 5 before presenting the theorem of correctness of surface effects in Section 7. The small-step finite-prefix safety layer is summarized in Section 8. We conclude in Section 10 after discussion on related work.

1 Motivating Example

The ML-like program given in Fig. 2 illustrates the intuition behind surface effects and their programmatic expressiveness. Let us first consider the function `incr` which simply increments a number stored in a mutable reference. The type of the argument is $\mathsf{ref}\,(\mathsf{int})\,\mathsf{at}\,\rho$ and the latent static effect of the function body is $\{\mathsf{R}(\rho), \mathsf{W}(\rho)\}$, since the reference is both read and written. The corresponding

effect function `incr_eff` describes `incr` by means of the surface-effect expression $\widehat{\mathsf{Read}}(r) \oplus \widehat{\mathsf{Write}}(r)$, which evaluates at run-time to a concretized, and more precise, version of the static effect: instead of whole regions, the computed effects mention the concrete addresses ($l@r$) accessed inside the region allocated for ρ.

```
fun incr (r : ref (int) at $\rho$) =
  { set r (get r + 1) (* computational term *)
  | incr_eff r } (* effect term *)

fun incr_eff (r : ref (int) at $\rho$) =
  Read(r) $\oplus$ Write(r) (* surface-effect summary *)
```

Figure 2: Motivating example: a function paired with its surface-effect summary.

Informally, if `incr_eff` correctly describes `incr`, then the relation ($\mathsf{incr} \blacktriangleleft \mathsf{incr\_eff}$) must hold statically. The type of $\{\mathsf{incr} \mid \mathsf{incr\_eff}\}$ is $\{\mathsf{unit} \,!\, \{\mathsf{R}(\rho), \mathsf{W}(\rho)\} \mid \mathsf{eff} \,!\, \emptyset\}$. This type expression assigns the type unit and the effect $\{\mathsf{R}(\rho), \mathsf{W}(\rho)\}$ to `incr`, and the type eff and effect $\emptyset$ to `incr_eff`. In this way, in order to prove $e_b \blacktriangleleft e_e$, it is necessary that the effect application of `incr_eff` produces an effect summary that correctly describes the latent effect of `incr`.

Although the mutable reference r is resolved to a concrete region/location pair only at run time, summary soundness is established statically. The mechanization uses two distinct conservative relations rather than a single formal order $\epsilon \sqsupseteq \Theta \sqsupseteq \phi$: static-effect soundness relates static actions to dynamic traces, while surface-effect correctness establishes $\phi \sqsubseteq \Theta$ for the evaluated summary. In the motivating case, two calls to `incr` on distinct concrete addresses are therefore distinguishable by their computed summaries even when their static effects mention the same region.

2 Static, Dynamic and Surface Effects

Four objects must be kept distinct throughout the development:

- **Static effects** (ϵ) are compile-time denotations used in type-and-effect judgments $\Gamma \vdash e : \tau \,!\, \epsilon$. They abstract memory accesses at region granularity, e.g. $\mathsf{R}(\rho)$, $\mathsf{W}(\rho)$, and $\mathsf{A}(\rho)$.
- **Dynamic traces** (ϕ) are run-time traces produced by continuation-machine executions. Their elements record concrete allocation, read, and write actions at region/location pairs.
- **Surface-effect terms** ($\widehat{\Theta}$) are first-class expressions of the effect-summary mini-language. They may contain ordinary expressions whose run-time values determine the summary.
- **Computed effects** (Θ) are values obtained by evaluating surface-effect terms. A computed effect is either $\top$ or a set of abstract region actions and/or concrete region/location actions.

The mechanization defines $\top$, the empty computed effect, and union of computed effects, but does not introduce or prove a lattice/precision-order structure across ϵ, Θ, and ϕ. Accordingly, we use $\phi \sqsubseteq \Theta$ only for the explicit trace-to-computed-effect soundness relation proved in Section 7.

3 Syntax

A "computational" expression, e_b, can be soundly described by an "effect" expression, e_e, written as $\{e_b \mid e_e\}$. Expressions include variables, recursive functions and recursive region abstractions, computational applications (capp), surface effect applications (eapp), region applications, arithmetic and boolean expressions, imperative functions on mutable references, annotated parallel tuples $(\!|\, \_,\ \_ \,|\!)$, and surface effects ($\widehat{\Theta}$). In the recursive function $\mu f.\lambda x.\{e_b \mid e_e\}$, the variable x is bound in both expressions e_b and e_e. The correctness relation between e_b and e_e is statically defined by ($\Gamma \vdash e_b \blacktriangleleft e_e$), as will be presented in Section 7.

4 Operational Semantics

The paper-facing operational semantics is a continuation-machine semantics. It is call-by-value and emits labels during execution. Silent labels record administrative motion through the continuation stack; action labels record concrete allocation, read, and write events. Finite executions therefore cover both terminating runs and finite prefixes of runs that may diverge.

Machine domains.

$$
\begin{aligned}
E \in \mathit{Env} &\triangleq \mathit{Variables} \rightharpoonup \mathit{Values} && \text{(variable assignment)}\\
P \in \mathit{Rgn} &\triangleq \mathit{RgnVars} \rightharpoonup \mathit{RgnConst} && \text{(regions in scope)}\\
H \in \mathit{Heap} &\triangleq \mathit{Addr} \rightharpoonup \mathit{Values} && \text{(heap assignment)}\\
\langle E, P, e\rangle \in \mathit{Closures} &\triangleq \mathit{Env} \times \mathit{Rgn} \times \mathit{Expressions} && \text{(function closures)}\\
v \in \mathit{Values} &\triangleq \mathbb{N} + \mathbb{B} + \mathit{Addr} + \mathit{Closures} + \Theta + () + (v, v) && \text{(dynamic values)}
\end{aligned}
$$

Machine states and labels are:

$$
\begin{aligned}
s &::= \langle H, E, P, e, k\rangle \mid \langle H, v, k\rangle \mid \mathsf{done}(H, v),\\
\ell &::= \tau \mid \mathsf{act}(a),\\
a &::= \mathsf{Alloc}(l@r, v) \mid \mathsf{Read}(l@r, v) \mid \mathsf{Write}(l@r, v).
\end{aligned}
$$

The initial state for expression e is $\mathsf{init}(H, E, P, e) = \langle H, E, P, e, \mathsf{doneK}\rangle$. The one-step relation is written $s \xrightarrow{\ell} s'$, and its reflexive transitive closure is written $s \xrightarrow{\alpha*} s'$, where α is the list of emitted dynamic actions. We write $\Phi(\alpha)$ for the structured trace obtained from the action list α, and $s \xrightarrow{\Phi} s'$ when a structured trace is used directly.

Continuations contain one frame per evaluation context. The frames used below include application frames $K^{\mu}_{f}(e_a, E, P, k)$ and $K^{\mu}_{a}(E, P, f, x, e_b, e_s, k)$ for computational application, analogous frames K^{e}_{f} and K^{e}_{a} for effect application, heap-operation frames for allocation, dereference, and assignment, and parallel frames

$$K^{par}_{1}(e_{f1}, e_{a1}, e_{f2}, e_{a2}, E, P, k), \quad K^{par}_{2}(e_{f1}, e_{a1}, e_{f2}, e_{a2}, E, P, \Theta_1, k),$$

$K^{par}_{\mu 1}(e_{f2}, e_{a2}, E, P, k)$, and $K^{par}_{\mu 2}(v_1, k)$.

Representative small-step rules.

$$\textsc{S-Const}\ \frac{}{\langle H, E, P, \mathsf{const}\ n, k\rangle \xrightarrow{\tau} \langle H, n, k\rangle} \qquad \textsc{S-Var}\ \frac{E(x) = v}{\langle H, E, P, x, k\rangle \xrightarrow{\tau} \langle H, v, k\rangle} \qquad \textsc{S-Done}\ \frac{}{\langle H, v, \mathsf{doneK}\rangle \xrightarrow{\tau} \mathsf{done}(H, v)}$$

$$\textsc{S-Clos}\ \frac{\varphi = \mu f.\lambda x.\{e_b \mid e_s\} \ \vee\ \varphi = \Lambda\rho.e}{\langle H, E, P, \varphi, k\rangle \xrightarrow{\tau} \langle H, \langle E, P, \varphi\rangle, k\rangle} \qquad \textsc{S-Capp-Fun}\ \frac{}{\langle H, E, P, \mathsf{capp}\ e_f\ e_a, k\rangle \xrightarrow{\tau} \langle H, E, P, e_f, K_f^{\mu}(e_a, E, P, k)\rangle}$$

$$\textsc{S-Capp-Arg}\ \frac{v_f = \langle E', P', \mu f.\lambda x.\{e_b \mid e_s\}\rangle}{\langle H, v_f, K_f^{\mu}(e_a, E, P, k)\rangle \xrightarrow{\tau} \langle H, E, P, e_a, K_a^{\mu}(E', P', f, x, e_b, e_s, k)\rangle}$$

$$\textsc{S-Capp-Body}\ \frac{E'' = E'[f \mapsto v_f][x \mapsto v_a] \qquad v_f = \langle E', P', \mu f.\lambda x.\{e_b \mid e_s\}\rangle}{\langle H, v_a, K_a^{\mu}(E', P', f, x, e_b, e_s, k)\rangle \xrightarrow{\tau} \langle H, E'', P', e_b, k\rangle}$$

$$\textsc{S-Eapp-Fun}\ \frac{}{\langle H, E, P, \mathsf{eapp}\ e_f\ e_a, k\rangle \xrightarrow{\tau} \langle H, E, P, e_f, K_f^{e}(e_a, E, P, k)\rangle}$$

$$\textsc{S-Eapp-Arg}\ \frac{v_f = \langle E', P', \mu f.\lambda x.\{e_b \mid e_s\}\rangle}{\langle H, v_f, K_f^{e}(e_a, E, P, k)\rangle \xrightarrow{\tau} \langle H, E, P, e_a, K_a^{e}(E', P', f, x, e_b, e_s, k)\rangle}$$

$$\textsc{S-Eapp-Body}\ \frac{E'' = E'[f \mapsto v_f][x \mapsto v_a] \qquad v_f = \langle E', P', \mu f.\lambda x.\{e_b \mid e_s\}\rangle}{\langle H, v_a, K_a^{e}(E', P', f, x, e_b, e_s, k)\rangle \xrightarrow{\tau} \langle H, E'', P', e_s, k\rangle}$$

$$\textsc{S-Alloc}\ \frac{P(p) = r \qquad \mathsf{fresh}(H, r) = l}{\langle H, v, K^{ref}(p, P, k)\rangle \xrightarrow{\mathsf{act}(\mathsf{Alloc}(l@r,v))} \langle H[(r, l) \mapsto v], l@r, k\rangle}$$

$$\textsc{S-Get}\ \frac{P(p) = r \qquad H(r, l) = v}{\langle H, l@r, K^{get}(p, P, k)\rangle \xrightarrow{\mathsf{act}(\mathsf{Read}(l@r,v))} \langle H, v, k\rangle}$$

$$\textsc{S-Set}\ \frac{P(p) = r \qquad H(r, l) \neq \bot}{\langle H, v, K^{set}(p, l, P, k)\rangle \xrightarrow{\mathsf{act}(\mathsf{Write}(l@r,v))} \langle H[(r, l) \mapsto v], (), k\rangle}$$

Evaluation of surface effects. Surface-effect expressions are ordinary expressions whose values are computed effects. Abstract summaries return region-level actions without touching the heap; concrete summaries first evaluate their argument and then return a concrete computed action.

S-Abst-Read
$$\frac{P(p) = r}{\langle H, E, P, \widehat{\mathsf{R}}(p), k\rangle \xrightarrow{\tau} \langle H, \mathsf{R}(r), k\rangle}$$

S-Abst-Write
$$\frac{P(p) = r}{\langle H, E, P, \widehat{\mathsf{W}}(p), k\rangle \xrightarrow{\tau} \langle H, \mathsf{W}(r), k\rangle}$$

S-Abst-Alloc
$$\frac{P(p) = r}{\langle H, E, P, \widehat{\mathsf{A}}(p), k\rangle \xrightarrow{\tau} \langle H, \mathsf{A}(r), k\rangle}$$

S-Conc-Read
$$\frac{}{\langle H, l@r, K^{read}(k)\rangle \xrightarrow{\tau} \langle H, \mathsf{Read}(l@r), k\rangle}$$

S-Conc-Write
$$\frac{}{\langle H, l@r, K^{write}(k)\rangle \xrightarrow{\tau} \langle H, \mathsf{Write}(l@r), k\rangle}$$

S-Concat
$$\frac{}{\langle H, \Theta_2, K^{concat}(\Theta_1, k)\rangle \xrightarrow{\tau} \langle H, \Theta_1 \cup \Theta_2, k\rangle}$$

S-Top
$$\frac{}{\langle H, E, P, \mathsf{Top}, k\rangle \xrightarrow{\tau} \langle H, \top, k\rangle}$$

S-Empty
$$\frac{}{\langle H, E, P, \mathsf{Pure}, k\rangle \xrightarrow{\tau} \langle H, \emptyset, k\rangle}$$

Implicit-parallel tuples. The syntax contains a single staged tuple constructor $(\!|\, e_{f1}\, e_{a1},\ e_{f2}\, e_{a2} \,|\!)$. The machine first evaluates the two effect applications and obtains computed summaries Θ_1 and Θ_2:

S-Par-Eff1
$$\frac{}{\langle H, E, P, (\!|\, e_{f1}\, e_{a1},\ e_{f2}\, e_{a2} \,|\!), k\rangle \xrightarrow{\tau} \langle H, E, P, \mathsf{eapp}\ e_{f1}\, e_{a1}, K_1^{par}(e_{f1}, e_{a1}, e_{f2}, e_{a2}, E, P, k)\rangle}$$

S-Par-Eff2
$$\frac{}{\langle H, \Theta_1, K_1^{par}(e_{f1}, e_{a1}, e_{f2}, e_{a2}, E, P, k)\rangle \xrightarrow{\tau} \langle H, E, P, \mathsf{eapp}\ e_{f2}\, e_{a2}, K_2^{par}(e_{f1}, e_{a1}, e_{f2}, e_{a2}, E, P, \Theta_1, k)\rangle}$$

At that point the dynamic guard is

$$\mathsf{Pass}(\Theta_1, \Theta_2) \quad \triangleq \quad \mathsf{Disjoint}(\Theta_1, \Theta_2) \wedge \neg(\Theta_1 \otimes \Theta_2).$$

If the guard fails, the ordinary continuation machine follows the sequential fallback path:

$$\langle H, \Theta_2, K_2^{par}(\cdots, \Theta_1, k)\rangle \xrightarrow{\tau} \langle H, E, P, \mathsf{capp}\ e_{f1}\, e_{a1}, K_{\mu 1}^{par}(e_{f2}, e_{a2}, E, P, k)\rangle.$$

The same concrete continuation frame also represents the sequential prefix used after the check succeeds. The checked-interleaving behavior for successful checks is stated by a separate structured transition relation

$$s_{\|} \xrightarrow{\alpha}_{\|} s',$$

which runs the two computational applications from the same source heap and records their branch traces separately. The theorem statements in Section 8 connect this checked relation and the sequential fallback path to one uniform safety story. There is no separate source-level sequential tuple constructor.

Effect-summary evaluation is proved read-only from typing: if a well-typed summary execution emits a read-only trace, the final heap is the initial heap. A summary may *denote* an allocation through an abstract allocation action without performing an allocation while the summary itself is evaluated.

5 Disjointness of Evaluated Surface Effects

The mechanization distinguishes two predicates on computed effects. $\Theta_1 \otimes \Theta_2$ is existential: it holds when the sets contain a pair of computed actions in the conflict relation. In contrast, $\mathsf{Disjoint}(\Theta_1, \Theta_2)$ is universal: every action represented by the left summary must be disjoint from every action represented by the right summary. The dynamic tuple guard requires *both* disjointness and absence of a computed conflict. This distinction is important because scheduler safety uses disjointness to transport non-overlap to all dynamic actions and uses conflict absence to rule out the explicit dynamic read–write and write–write conflict constructors.

At concrete granularity, the intended disjointness test is inequality of the whole region/location pair, $(r_1, l_1) \neq (r_2, l_2)$; two different locations in the same region are therefore disjoint. Read–read pairs are always disjoint. When an abstract write or allocation action is involved, region inequality is required because the summary does not identify a concrete location. The distinguished value $\top$ conflicts with every computed effect and has no disjointness constructor, so it cannot enable the checked interleaving path.

Deciding disjointness.

$$\textsc{C-Read-Write}\ \frac{\theta = \mathsf{Read}(l@r) \lor \theta = \mathsf{Write}(l@r) \lor \theta = \mathsf{R}(r) \lor \theta = \mathsf{W}(r)}{\mathsf{W}(r) \square \theta \land \mathsf{Write}(l@r) \square \theta} \qquad \textsc{C-Ext-Theta}\ \frac{\theta_1 \in \Theta_1 \qquad \theta_2 \in \Theta_2 \qquad \theta_1 \square \theta_2}{\Theta_1 \otimes \Theta_2}$$

$$\textsc{D-Theta}\ \frac{\forall \theta_1 \in \Theta_1.\ \forall \theta_2 \in \Theta_2.\ \mathsf{DisjointAction}(\theta_1, \theta_2)}{\mathsf{Disjoint}(\Theta_1, \Theta_2)} \qquad \textsc{C-Top}\ \frac{}{\Theta \otimes \top}$$

The small-step progress and safety theorems assume that the guard is decidable:

$$\forall \Theta_1, \Theta_2.\ \mathsf{Pass}(\Theta_1, \Theta_2) \lor \neg \mathsf{Pass}(\Theta_1, \Theta_2).$$

The current development does not additionally certify a boolean implementation of the check by a reflection theorem.

6 Static Semantics

The typing rules for the implicitly parallel language are given in the present section. A variable context Γ is a total map that records free variables and their types and a region context Δ is a set of region variables in scope. The judgment $\Gamma; \Delta \vdash e : \tau \,!\, \epsilon$ asserts that expression e has type τ under Γ and Δ and the latent (static) effect ϵ. As declared in rule [T-ABST], the correctness between a computational–effect pair is denoted by the binary relation ($\blacktriangleleft$). The rule [T-CAPP] declares the return type of a computational application, capp, and the same reasoning is made for eapp (whose return type is always eff).

Because the mechanization uses logical equality when proving soundness properties about surface effects, types are represented in a locally nameless style (Charguéraud, 2012), so that quantified types are equivalent up to α-conversion. Bound variables are represented with de Bruijn indices and free-variable names are left unchanged. This representation lets substitution be stated directly by type-level β-reduction.

The mechanized predicate ReadOnly on static effects is generated by three cases: the empty effect is read-only, a singleton region-read effect is read-only, and unions of read-only effects are read-only. Effect soundness lifts this property to dynamic traces. The corresponding trace predicate contains only the empty trace, concrete read actions, and sequential or parallel compositions of read-only traces; allocation and write actions are excluded. Consequently, read-only summary evaluation preserves the heap. This is a semantic property, not a cost claim: the development does not prove that a read-only summary terminates or that evaluating it has constant overhead.

Corollary 1 (Evaluation of pure surface effects imply the empty trace)**.** *For all traces ϕ such that the semantic relation $[\![\phi]\!] \subseteq \emptyset$ holds, then $\phi = \mathsf{Nil}$.*

Lemma 1 (Read-only traces of dynamic effects imply no side-effects)**.** *Given that* ReadOnly(ϕ)*, a terminal structured execution*

$$\mathsf{init}(H, E, P, e) \xrightarrow{\phi} \mathsf{done}(H', v)$$

implies that $H \equiv H'$ up to key-value pairs.

Lemma 2 (Read-only static effects imply read-only traces of dynamic effects)**.** *For all static effects ϵ and traces of dynamic effects ϕ, if $[\![\phi]\!] \subseteq [\![\epsilon]\!]$ holds and* ReadOnly(ϵ) *holds for static effects, then* ReadOnly(ϕ) *also holds for traces of dynamic effects.*

7 Correctness of Surface Effects

The present section defines the static relation ($\blacktriangleleft$) between the syntax of expressions, e, and surface effects, $\widehat{\Theta}$, as they were previously defined in Section 3. Informally, given a set of syntax-directed inference rules, the correctness of surface effects implies that if $\Gamma; \Delta \vdash e \blacktriangleleft \widehat{\Theta}$, then the run-time effects of the corresponding terminal machine executions satisfy the relation $\phi \sqsubseteq \Theta$ for all heaps, run-time environments and allocated region maps. Note that the given set of rules is incomplete, in the sense that an infinite number of sound effect summaries can be used to describe a given expression. Moreover, some expressions can be over-described by 'Top' only.

Surface effect soundness.

$$\textsc{Surface-Read-1}\ \frac{}{\mathsf{Read}\, l@r \sqsubseteq \mathsf{Read}\, l@r} \qquad \textsc{Surface-Read-2}\ \frac{}{\mathsf{Read}\, l@r \sqsubseteq \mathsf{R}\, r} \qquad \textsc{Surface-Write-1}\ \frac{}{\mathsf{Write}\, l@r \sqsubseteq \mathsf{Write}\, l@r} \qquad \textsc{Surface-Write-2}\ \frac{}{\mathsf{Write}\, l@r \sqsubseteq \mathsf{W}\, r}$$

$$\textsc{Sound-Eff-Sets}\ \frac{\forall \iota \in [\![\phi]\!].\ \exists \theta \in \vartheta.\ \iota \sqsubseteq \theta}{\phi \sqsubseteq \vartheta} \qquad \textsc{Sound-Eff-Top}\ \frac{}{\phi \sqsubseteq \top}$$

The derived small-step theorem for effect application uses the following mathematical interpretation of an observed trace as a computed summary:

$$\begin{aligned}
\mathcal{T}(\mathsf{Nil}) &= \emptyset, \\
\mathcal{T}(\mathsf{Alloc}(l@r, v)) &= \{\mathsf{A}(r)\}, \\
\mathcal{T}(\mathsf{Read}(l@r, v)) &= \{\mathsf{Read}(l@r)\}, \\
\mathcal{T}(\mathsf{Write}(l@r, v)) &= \{\mathsf{Write}(l@r)\}, \\
\mathcal{T}(\phi_1; \phi_2) &= \mathcal{T}(\phi_1) \cup \mathcal{T}(\phi_2), \\
\mathcal{T}(\phi_1 \parallel \phi_2) &= \mathcal{T}(\phi_1) \cup \mathcal{T}(\phi_2).
\end{aligned}$$

Thus every structured trace approximates its trace-induced computed summary:

$$\phi \sqsubseteq \mathcal{T}(\phi).$$

For an effect application, the continuation machine evaluates the function position and the argument position before entering the paired summary body. If the observed summary trace decomposes as

$$\phi_e = \phi_f; \phi_a; \phi_s$$

and the summary body returns Θ_s, the observed summary used by the derived theorem is

$$\mathsf{Aug}(\phi_f, \phi_a, \Theta_s) \triangleq \mathcal{T}(\phi_f) \cup \mathcal{T}(\phi_a) \cup \Theta_s.$$

This is an interpretation layer over terminal runs: the machine still returns the body value Θ_s, while the proof can also reason about the function-position and argument-position traces that were required to reach the body.

Because 'Top' is a programming construct, the approximation relation $\phi \sqsubseteq \top$ is trivially sound for every trace. Its scheduler behavior is deliberately conservative: $\top$ conflicts with every computed effect and cannot satisfy $\mathsf{Disjoint}$. Hence a 'Top' summary does not justify the checked-interleaving path. Evaluating 'Top' itself produces the empty dynamic trace and preserves the heap, but the formal development contains no complexity model from which a constant-overhead claim could be derived.

Correctness inference rules ($\Gamma; \Delta; P \vdash e \blacktriangleleft \widehat{\Theta}$). The judgment is indexed by the run-time region map P. For a static effect ϵ, write $\epsilon[P]$ for the effect obtained by substituting the region values in P. The following rules give the inductive presentation used in the paper; in particular, pure constants, booleans, variables, abstractions, and region abstractions are described by Pure, rather than by an arbitrary summary.

$$\textsc{Eff-Const}\ \frac{\Gamma;\Delta \vdash \mathsf{const}\ n : \mathsf{nat}\,!\,\emptyset}{\Gamma;\Delta;P \vdash \mathsf{const}\ n \blacktriangleleft \mathsf{Pure}} \qquad \textsc{Eff-Bool}\ \frac{\Gamma;\Delta \vdash \mathsf{bool}\ b : \mathsf{bool}\,!\,\emptyset}{\Gamma;\Delta;P \vdash \mathsf{bool}\ b \blacktriangleleft \mathsf{Pure}} \qquad \textsc{Eff-Var}\ \frac{\Gamma;\Delta \vdash \mathsf{var}\ x : \tau\,!\,\emptyset}{\Gamma;\Delta;P \vdash \mathsf{var}\ x \blacktriangleleft \mathsf{Pure}}$$

$$\textsc{Eff-Abst}\ \frac{}{\Gamma;\Delta;P \vdash \mu f.\lambda x.\{e_b \mid e_e\} \blacktriangleleft \mathsf{Pure}} \qquad \textsc{Eff-Rgn-Abst}\ \frac{}{\Gamma;\Delta;P \vdash \lambda\rho.\, e \blacktriangleleft \mathsf{Pure}}$$

$$\textsc{Eff-App}\ \frac{\begin{array}{c}\Gamma;\Delta \vdash \mathsf{capp}\ e_f\ e_a : \tau_b\,!\,\epsilon_b \qquad \Gamma;\Delta \vdash \mathsf{eapp}\ e_f\ e_a : \mathsf{eff}\,!\,\epsilon_e \\ \mathsf{ReadOnly}(\epsilon_e[P]) \qquad \Gamma;\Delta \vdash e_f : \tau_f\,!\,\epsilon_f \qquad \Gamma;\Delta \vdash e_a : \tau_a\,!\,\epsilon_a \qquad \Gamma;\Delta;P \vdash e_f \blacktriangleleft (\mathsf{eapp}\ e_f\ e_a) \\ \Gamma;\Delta;P \vdash e_a \blacktriangleleft (\mathsf{eapp}\ e_f\ e_a) \qquad \mathsf{ReadOnly}(\epsilon_f[P]) \qquad \mathsf{ReadOnly}(\epsilon_a[P])\end{array}}{\Gamma;\Delta;P \vdash \mathsf{capp}\ e_f\ e_a \blacktriangleleft (\mathsf{eapp}\ e_f\ e_a)}$$

$$\textsc{Eff-Par}\ \frac{\begin{array}{c}\Gamma;\Delta \vdash \mathsf{eapp}\ e_{f1}\ e_{a1} : \mathsf{eff}\,!\,\epsilon_{e1} \qquad \mathsf{ReadOnly}(\epsilon_{e1}[P]) \\ \Gamma;\Delta \vdash \mathsf{eapp}\ e_{f2}\ e_{a2} : \mathsf{eff}\,!\,\epsilon_{e2} \qquad \mathsf{ReadOnly}(\epsilon_{e2}[P]) \qquad \Gamma;\Delta;P \vdash \mathsf{eapp}\ e_{f1}\ e_{a1} \blacktriangleleft e_1 \\ \Gamma;\Delta;P \vdash \mathsf{eapp}\ e_{f2}\ e_{a2} \blacktriangleleft e_2 \qquad \Gamma;\Delta;P \vdash \mathsf{capp}\ e_{f1}\ e_{a1} \blacktriangleleft e_3 \qquad \Gamma;\Delta;P \vdash \mathsf{capp}\ e_{f2}\ e_{a2} \blacktriangleleft e_4\end{array}}{\Gamma;\Delta;P \vdash (\!|\ \mathsf{capp}\ e_{f1}\ e_{a1},\ \mathsf{capp}\ e_{f2}\ e_{a2}\ |\!) \blacktriangleleft (e_1 \oplus e_2) \oplus (e_3 \oplus e_4)}$$

$$\textsc{Eff-Rgn-App}\ \frac{\Gamma;\Delta \vdash e_r : \tau\,!\,\epsilon \qquad \Gamma;\Delta \vdash \mathsf{Pure} : \mathsf{eff}\,!\,\emptyset \qquad \Gamma;\Delta;P \vdash e_r \blacktriangleleft \mathsf{Pure}}{\Gamma;\Delta;P \vdash e_r\,[r] \blacktriangleleft \mathsf{Pure}}$$

$$\textsc{Eff-Cond}\ \frac{\begin{array}{c}\Gamma;\Delta \vdash e_c : \tau_c\,!\,\epsilon_c \qquad \Gamma;\Delta \vdash e_t : \tau_t\,!\,\epsilon_t \qquad \Gamma;\Delta \vdash e_f : \tau_f\,!\,\epsilon_f \\ \mathsf{ReadOnly}(\epsilon_c[P]) \qquad \Gamma;\Delta;P \vdash e_c \blacktriangleleft \mathsf{Pure} \qquad \Gamma;\Delta;P \vdash e_t \blacktriangleleft e_1 \qquad \Gamma;\Delta;P \vdash e_f \blacktriangleleft e_2\end{array}}{\Gamma;\Delta;P \vdash \mathsf{cond}\ e_c\ e_t\ e_f \blacktriangleleft \mathsf{cond}\ e_c\ e_1\ e_2}$$

$$\textsc{Eff-Alloc}\ \frac{\Gamma;\Delta \vdash e : \tau\,!\,\epsilon \qquad \Gamma;\Delta;P \vdash e \blacktriangleleft e_1}{\Gamma;\Delta;P \vdash \mathsf{alloc}\ r\ e \blacktriangleleft e_1 \oplus \widehat{\mathsf{A}}(r)} \qquad \textsc{Eff-Get-Abs}\ \frac{\Gamma;\Delta \vdash e : \tau\,!\,\epsilon \qquad \Gamma;\Delta;P \vdash e \blacktriangleleft e_1}{\Gamma;\Delta;P \vdash \mathsf{get}\ p\ e \blacktriangleleft e_1 \oplus \widehat{\mathsf{R}}(p)}$$

$$\textsc{Eff-Get-Conc}\ \frac{\Gamma;\Delta \vdash e : \tau\,!\,\epsilon \qquad \Gamma;\Delta;P \vdash e \blacktriangleleft \mathsf{Pure}}{\Gamma;\Delta;P \vdash \mathsf{get}\ r\ e \blacktriangleleft \widehat{\mathsf{Read}}(e)}$$

$$\textsc{Eff-Set-Abs}\ \frac{\Gamma;\Delta;P \vdash e_r \blacktriangleleft e_1 \qquad \Gamma;\Delta;P \vdash e_v \blacktriangleleft e_2 \qquad \Gamma;\Delta \vdash e_r : \tau_r\,!\,\epsilon_r \qquad \mathsf{ReadOnly}(\epsilon_r[P])}{\Gamma;\Delta;P \vdash \mathsf{set}\ p\ e_r\ e_v \blacktriangleleft e_1 \oplus (e_2 \oplus \widehat{\mathsf{W}}(p))}$$

$$\textsc{Eff-Set-Conc}\ \frac{\Gamma;\Delta;P \vdash e_r \blacktriangleleft e_1 \qquad \Gamma;\Delta;P \vdash e_v \blacktriangleleft e_2 \qquad \Gamma;\Delta \vdash e_r : \tau_r\,!\,\epsilon_r \qquad \mathsf{ReadOnly}(\epsilon_r[P])}{\Gamma;\Delta;P \vdash \mathsf{set}\ r\ e_r\ e_v \blacktriangleleft e_1 \oplus (e_2 \oplus \widehat{\mathsf{Write}}(e_r))}$$

$$\textsc{Eff-Arith}\ \frac{\begin{array}{c}\Gamma;\Delta \vdash e_1 : \tau_1\,!\,\epsilon_1 \\ \Gamma;\Delta \vdash e_2 : \tau_2\,!\,\epsilon_2 \qquad \mathsf{ReadOnly}(\epsilon_1[P]) \qquad \Gamma;\Delta;P \vdash e_1 \blacktriangleleft \widehat{\Theta}_1 \qquad \Gamma;\Delta;P \vdash e_2 \blacktriangleleft \widehat{\Theta}_2\end{array}}{\Gamma;\Delta;P \vdash e_1 \odot e_2 \blacktriangleleft \widehat{\Theta}_1 \oplus \widehat{\Theta}_2}$$

$$\textsc{Eff-Top}\ \frac{}{\Gamma;\Delta;P \vdash e \blacktriangleleft \mathsf{Top}}$$

Here $\odot \in \{+, -, \times, =\}$ abbreviates the four corresponding arithmetic and equality cases. The concrete read and write rules above use a concrete region value r.

Theorem 1 (Correctness of surface effects)**.** *Assume well-formed region, typing, heap-typing, and*

environment-typing judgments for H, E, P, and e_b. If the computation and summary are related by $\Gamma; \Delta; P \vdash e_b \blacktriangleleft e_e$, and

$$\mathsf{init}(H, E, P, e_b) \xrightarrow{\phi_b} \mathsf{done}(H_b, v_b) \qquad \textit{and} \qquad \mathsf{init}(H, E, P, e_e) \xrightarrow{\phi_e} \mathsf{done}(H_e, \mathsf{eff}(\Theta_e)),$$

with $\mathsf{ReadOnly}(\phi_e)$*, then*

$$\phi_b \sqsubseteq \Theta_e.$$

The conclusion of the correctness theorem is approximation only. In particular, it does *not* conclude that the computation trace ϕ_b is read-only. The small-step results in Section 8 reuse this approximation story while stating finite-prefix and checked/fallback safety directly over the continuation machine.

Corollary 2 (Read-only, heap-preserving summary evaluation)**.** *For a well-typed computation/effect application pair, static effect soundness and the typing premises of the correctness relation imply* $\mathsf{ReadOnly}(\phi_e)$*. Therefore summary evaluation preserves the initial heap: $H \equiv H_e$.*

The proof separates two facts: the computation trace is shown to satisfy $\phi_b \sqsubseteq \Theta_e$, while the trace produced by evaluating the effect application is separately shown to be read-only. Thus a computed action such as $\mathsf{A}(r)$ may *describe* a future allocation without the evaluation of the summary itself allocating memory.

Theorem 2 (Augmented effect-application summary)**.** *Consider a terminal evaluation of an effect application:*

$$\mathsf{init}(H, E, P, \mathsf{eapp}\ e_f\ e_a) \xrightarrow{\phi_e} \mathsf{done}(H_e, \mathsf{eff}(\Theta_s)).$$

Suppose the run decomposes through the ordinary effect-application frames as

$$\mathsf{init}(H, E, P, e_f) \xrightarrow{\phi_f} \mathsf{done}(H_f, \langle E', P', \mu f.\lambda x.\{e_b \mid e_s\}\rangle),$$
$$\mathsf{init}(H_f, E, P, e_a) \xrightarrow{\phi_a} \mathsf{done}(H_a, v_a),$$
$$\mathsf{init}(H_a, E'[f \mapsto v_f][x \mapsto v_a], P', e_s) \xrightarrow{\phi_s} \mathsf{done}(H_e, \mathsf{eff}(\Theta_s)).$$

If the body-summary trace is covered by the body summary, $\phi_s \sqsubseteq \Theta_s$, then the whole observed effect-application trace is covered by the augmented observed summary:

$$\phi_e \sqsubseteq \mathsf{Aug}(\phi_f, \phi_a, \Theta_s).$$

The theorem does not change the operational semantics: the terminal value of the effect application is still $\mathsf{eff}(\Theta_s)$. It records an additional interpretation of the observed run that charges the traces needed to evaluate e_f and e_a to $\mathcal{T}(\phi_f)$ and $\mathcal{T}(\phi_a)$ rather than requiring the body summary Θ_s alone to cover them.

Theorem 3 (Scheduler soundness)**.** *Let $\phi_1 \sqsubseteq \Theta_1$ and $\phi_2 \sqsubseteq \Theta_2$. If*

$$\mathsf{Disjoint}(\Theta_1, \Theta_2) \qquad \textit{and} \qquad \neg(\Theta_1 \otimes \Theta_2),$$

and $\mathsf{Det}(\phi_1)$ *and* $\mathsf{Det}(\phi_2)$*, then*

$$\mathsf{Det}(\phi_1 \parallel \phi_2).$$

The scheduler bridge first shows that a dynamic read–write or write–write conflict would induce a corresponding computed conflict, contradicting the summary-level premise. It separately transports computed-action disjointness to every cross-trace pair of dynamic actions. The result is the deterministic parallel trace judgment above.

Combining the preceding results gives the proof dependency used by the checked interleaving path:

verified computation/summary pair

$$\Downarrow$$
$$\phi_i \sqsubseteq \Theta_i$$
$$\Downarrow\ \mathsf{Disjoint}(\Theta_1, \Theta_2) \wedge \neg(\Theta_1 \otimes \Theta_2)$$
$$\mathsf{Det}(\phi_1 \parallel \phi_2)$$
$$\Downarrow$$

equivalent final heaps under terminating trace replay.

Runtime definitions.

$$\Sigma \triangleq \mathit{Addr} \rightharpoonup \mathit{Type} \qquad \text{(heap type assignment)}$$

$$\text{T-Heap}\ \frac{\forall l@r \in H,\ \Sigma \vdash H(l@r) : \Sigma(l@r)}{\star \vdash H : \Sigma} \qquad \text{T-Heap-Empty}\ \frac{}{\Sigma \vdash \emptyset : \emptyset} \qquad \text{T-Heap-Ext}\ \frac{\Sigma \vdash E : \Gamma \qquad \Sigma \vdash v : \tau}{\Sigma \vdash E, [x \mapsto v] : \Gamma, v : \tau}$$

As previously mentioned, the static relation ($\blacktriangleleft$) is intentionally incomplete: many sound summaries may describe the same computation, while some computations may be related only to a coarse summary by the supplied inductive rules. Concrete computed actions can refine a region-level distinction and may therefore satisfy the scheduler premises even when a region-only comparison would be inconclusive. 'Top', by contrast, is a sound approximation of every trace but cannot satisfy the disjointness/non-conflict premises of the dynamic guard. The current calculus contains no separate static-fallback rule for 'Top'; its role in the mechanized semantics is to conservatively prevent the checked interleaving path from being justified.

For example, the main purpose of designing a type system where the functional type contains a return type for a computational expression and a return type for an effect expression is to be able to describe any computational application ($\mathsf{capp}\ e_f\ e_a$) by its corresponding effect application ($\mathsf{eapp}\ e_f\ e_a$). The interesting aspect of assigning the "extended" functional type to the expression e_f is that it may carry a non-empty latent effect for the computational term which can be soundly described by the evaluation of the corresponding "surface" effect term. The returned computed summary of $\mathsf{eapp}\ e_f\ e_a$ is produced by the function body's summary expression. The observed run of the effect application may also contain read-only traces from evaluating e_f and e_a before the body is reached; the augmented-summary interpretation above accounts for those traces as $\mathcal{T}(\phi_f) \cup \mathcal{T}(\phi_a)$ without changing the value returned by the machine.

Another example is the expression e_b whose reference type is parametrized by a region p. Two different effect summaries are proven sound: the first is as much precise as the static effect would be (rule [EFF-GET]) and the second is potentially "much" more precise than the static effect (rule [EFF-GET-CONC]). The type of the argument of get is the reference type parametrized by a region variable or a region constant. Given the region p, the rule [EFF-GET] applies by chaining the effect summary of the argument with the static surface term $\widehat{\mathsf{R}}(p)$. When evaluated, this term yields a value that is equivalent to a de facto static effect. However, we can precisely describe a get operation using the surface term $\widehat{\mathsf{Read}}(e)$, which directly takes the argument as input in order to evaluate to a concrete memory address inside some allocated region.

The precision of the surface effects can be evaluated by analyzing their dynamic semantics. Consider, for example, the rules [S-Abst-Read] and [S-Conc-Write] in Section 4. The first corresponds to the evaluation of the "static version" of the surface effect for the get operation and the second corresponds to the evaluation of the "concrete version" of the surface effect for the set operation. The second rule shows that a concrete address, $l@r$, is being evaluated. Now consider the rules for conflict decision presented in Section 5. When concrete allocated addresses are known, it is possible to decide that the effects Read and Write do not conflict if the addresses are different even though they belong to the same region, allowing more precision (hence the motivating example presented in Fig. 2). However, if only the region is known, the "static version" of surface effects R and W don't allow the conflict decision at run-time to be more precise than the static analysis.

8 Small-Step Runtime Safety

The continuation machine supports safety statements over finite executions, not only over completed runs. The ordinary small-step safety invariant is indexed by a store typing Σ. Along a finite trace, allocation may extend Σ; preservation therefore produces a final store typing Σ' with $\Sigma \subseteq \Sigma'$. We write $s \xrightarrow{\alpha*} s'$ for a finite small-step execution emitting the dynamic-action list α, $\mathsf{init}(H, E, P, e)$ for the initial machine state, and $\mathsf{done}(H, v)$ for a terminal state. The judgment $\Sigma \vdash_{\mathsf{st}} s : \tau$ abbreviates the mechanized typed-state invariant, and $\Sigma \vdash_{\phi} \alpha$ states that the structured trace $\Phi(\alpha)$ is well typed with respect to Σ. The predicate $\mathsf{NotStuck}(s)$ means that s is either terminal or has a next small-step

transition, and $\mathsf{Shape}_{\Sigma}(v, \tau)$ records that the run-time form of v matches type τ under store typing Σ. The dynamic tuple guard is assumed decidable:

$$\forall \Theta_1, \Theta_2.\ \mathsf{Pass}(\Theta_1, \Theta_2) \vee \neg \mathsf{Pass}(\Theta_1, \Theta_2),$$

where

$$\mathsf{Pass}(\Theta_1, \Theta_2) \triangleq \mathsf{Disjoint}(\Theta_1, \Theta_2) \wedge \neg(\Theta_1 \otimes \Theta_2).$$

Theorem 4 (Finite-prefix safety)**.** *Assume that the heap, region environment, variable environment, and expression are well typed:*

$$\Sigma \vdash H \qquad \Delta \vdash P \qquad \Sigma; P \vdash E : \Gamma \qquad \Gamma; \Delta \vdash e : \tau \,!\, \epsilon.$$

If

$$\mathsf{init}(H, E, P, e) \xrightarrow{\alpha *} s,$$

then there exists an extended store typing Σ' such that

$$\Sigma \subseteq \Sigma' \quad \wedge \quad \Sigma' \vdash_{\mathsf{st}} s : \tau[P] \quad \wedge \quad \mathsf{NotStuck}(s) \quad \wedge \quad \Sigma' \vdash_{\phi} \alpha.$$

Theorem 5 (Terminal type and trace soundness)**.** *Under the same heap, environment, and expression typing assumptions, if*

$$\mathsf{init}(H, E, P, e) \xrightarrow{\alpha *} \mathsf{done}(H', v),$$

then there exists an extended store typing Σ' such that

$$\Sigma \subseteq \Sigma' \quad \wedge \quad \Sigma' \vdash H' \quad \wedge \quad \Sigma' \vdash v : \tau[P] \quad \wedge \quad \mathsf{Shape}_{\Sigma'}(v, \tau[P]) \quad \wedge \quad \Sigma' \vdash_{\phi} \alpha.$$

For implicit-parallel tuples, the small-step semantics exposes the dynamic effect check as an explicit control point. After both effect applications have produced computed summaries Θ_1 and Θ_2, the safety theorem splits on $\mathsf{Pass}(\Theta_1, \Theta_2)$. If the check succeeds, the checked interleaving relation for the two computational applications is trace safe. If the check fails, the ordinary continuation machine takes the sequential fallback path and that path is trace safe.

Theorem 6 (Checked-or-fallback finite-prefix safety)**.** *Assume the heap, region environment, variable environment, both computational applications, and the outer continuation are well typed:*

$$\Sigma \vdash H \qquad \Delta \vdash P \qquad \Sigma; P \vdash E : \Gamma$$
$$\Gamma; \Delta \vdash \mathsf{capp}\ e_{f1}\, e_{a1} : \tau_1 \,!\, \epsilon_1 \qquad \Gamma; \Delta \vdash \mathsf{capp}\ e_{f2}\, e_{a2} : \tau_2 \,!\, \epsilon_2$$
$$\Sigma \vdash k : (\tau_1[P] \times \tau_2[P]) \Rightarrow \tau_o.$$

Let s_{chk} be the checked-interleaving start state and s_{seq} be the sequential fallback start state for

$$(\!|\, \mathsf{capp}\ e_{f1}\, e_{a1},\ \mathsf{capp}\ e_{f2}\, e_{a2} \,|\!).$$

Then

$$\big(\mathsf{Pass}(\Theta_1, \Theta_2) \wedge \mathsf{Safe}^{\|}_{\Sigma, \tau_o}(s_{\mathsf{chk}})\big)$$
$$\vee$$
$$\big(\neg \mathsf{Pass}(\Theta_1, \Theta_2) \wedge \exists \Sigma'.\ \Sigma \subseteq \Sigma' \wedge \mathsf{Safe}_{\Sigma', \tau_o}(s_{\mathsf{seq}})\big).$$

Here $\mathsf{Safe}_{\Sigma, \tau}(s)$ is ordinary finite-prefix safety for a state, and $\mathsf{Safe}^{\|}_{\Sigma, \tau}(s)$ is its checked-interleaving analogue for paired branch states.

Theorem 7 (Checked-or-fallback terminal soundness)**.** *Under the same hypotheses, either the check succeeds and*

$$\begin{aligned} \forall \alpha, H', v.\ s_{\mathsf{chk}} &\xrightarrow{\alpha}_{\|} \mathsf{done}(H', v) \\ &\Longrightarrow \exists \Sigma'.\ \Sigma \subseteq \Sigma' \wedge \Sigma' \vdash H' \wedge \Sigma' \vdash v : \tau_o \\ &\wedge \mathsf{Shape}_{\Sigma'}(v, \tau_o) \wedge \Sigma' \vdash_{\phi} \alpha, \end{aligned}$$

or the check fails and

$$\begin{aligned} \forall \alpha, H', v.\ s_{\mathsf{seq}} &\xrightarrow{\alpha *} \mathsf{done}(H', v) \\ &\Longrightarrow \exists \Sigma'.\ \Sigma \subseteq \Sigma' \wedge \Sigma' \vdash H' \wedge \Sigma' \vdash v : \tau_o \\ &\wedge \mathsf{Shape}_{\Sigma'}(v, \tau_o) \wedge \Sigma' \vdash_{\phi} \alpha. \end{aligned}$$

The theorem is deliberately about the staged construct in the mechanized syntax. Since the syntax contains no distinct sequential tuple constructor, it is not an observational-equivalence theorem comparing two source-level tuple rules.

Structured terminal summary correctness. The paper-facing correctness theorem is stated directly over terminal continuation-machine runs. Suppose the two effect summaries for a parallel pair terminate as

$$\mathsf{init}(H, E, P, \mathsf{eapp}\ e_{f1}\ e_{a1}) \xrightarrow{\phi_{e1}} \mathsf{done}(H_{e1}, \mathsf{eff}(\Theta_1)),$$
$$\mathsf{init}(H, E, P, \mathsf{eapp}\ e_{f2}\ e_{a2}) \xrightarrow{\phi_{e2}} \mathsf{done}(H_{e2}, \mathsf{eff}(\Theta_2)),$$

and the dynamic check on Θ_1 and Θ_2 succeeds. Suppose also that the checked branch machine terminates with branch traces ϕ_{b1} and ϕ_{b2} and residual scheduler trace ϕ_p, so that the whole structured trace is

$$\phi = (\phi_{e1} \parallel \phi_{e2}); ((\phi_{b1} \parallel \phi_{b2}); \phi_p).$$

Then each computational branch admits a decomposition

$$\phi_{bi} = \phi_{fi}; \phi_{ai}; \phi_{si} \qquad (i \in \{1, 2\}),$$

where ϕ_{fi} is the function-position trace, ϕ_{ai} is the argument-position trace, and ϕ_{si} is the body trace. Under the usual heap, region, environment, and expression typing assumptions, these decompositions satisfy $\mathsf{ReadOnly}(\phi_{f1})$, $\mathsf{ReadOnly}(\phi_{a1})$, $\mathsf{ReadOnly}(\phi_{f2})$, and $\mathsf{ReadOnly}(\phi_{a2})$, and the terminal structured trace is covered by the augmented observed summary:

$$\phi \sqsubseteq \mathsf{Aug}(\phi_{e1}, \phi_{e2}, \mathcal{T}(\phi_{b1}) \cup \mathcal{T}(\phi_{b2}))\,.$$

Equivalently, expanding the two branch decompositions,

$$\phi \sqsubseteq \mathsf{Aug}(\phi_{e1}, \phi_{e2}, \mathsf{Aug}(\phi_{f1}, \phi_{a1}, \mathcal{T}(\phi_{si})) \cup \mathsf{Aug}(\phi_{f2}, \phi_{a2}, \mathcal{T}(\phi_{s2})))\,.$$

This theorem has no auxiliary replay, heap-agreement, or bounded-induction hypothesis: the extra coverage needed by function and argument evaluation is charged to the observed trace summaries via $\mathcal{T}$.

9 Discussion and Related Work

The closest static comparator is Deterministic Parallel Java (DPJ) (Bocchino et al., 2009). DPJ uses a modular compile-time type-and-effect system to guarantee deterministic semantics and expresses effects over regions. Surface effects retain static verification but move a value-dependent access summary into an executable source-level term. From the run-time side, classic inspector–executor work analyzes irregular dependences before the executor runs conflict-free work in parallel (Arenaz et al., 2004). The distinctive point of the present formulation is therefore not region effects or pre-execution access prediction in isolation, but the integration of a programmer-written, typed summary term with an approximation theorem and a scheduler-soundness proof.

This positioning also exposes a limitation. Expressive region/effect annotations can be difficult to write manually, and prior work has investigated inference precisely to reduce that burden (Tzannes et al., 2019). The present calculus assumes programmer-written surface-effect summaries and does not give an inference or synthesis algorithm. Annotation effort and the frequency with which summaries collapse to 'Top' are empirical questions left open here.

Particularly interesting in the context of this paper are the well-known hybrid platforms (Kawaguchi et al., 2012; Flanagan et al., 2006). The common denominator in this range of work is the use of precise and expressive specifications like refinement types. Liquid types is a type and effect system that provides the best of both static and dynamic approaches by allowing fine-multithreading in low-level languages with program-specific access patterns, at the same time that guarantees determinism. Effect specifications are expressed in first-order formulas using the decidable theory of linear arithmetic. In a more high-level object-oriented setting, the proto-language HOOL supports expressive object specifications, including subtyping, refinement types and object invariants. Similarly to our approach, refinements and invariants must be pure (i.e. mutable data is never updated during evaluation of effect specifications).

The design is hybrid in a precise sense. Verification of the computation/summary relation is static, while the scheduling guard is dynamic: the summary term is evaluated with run-time values and its computed effect is checked for disjointness and conflict absence before the checked interleaving

path is used. No failed dynamic cast changes the computational result, but summary evaluation is still run-time computation and may itself be input dependent. The continuation-machine metatheory proves finite-prefix safety for ordinary and checked/fallback executions and terminal soundness for completed machine runs. It does not prove termination of summary terms, bound their cost, or compare that cost with dynamic validation mechanisms. A future adequacy theorem could relate terminal machine runs to the archived terminating evaluator, but that bridge is not used as a paper theorem here.

Recent work on hybrid type and effect systems applied to concurrent programming is found in Long et al. (2013). The approach defines special static effects, designated open effects, that prevent object-oriented programs from being rejected when no static upper bound in the form of effect specification is available. More specifically, open effects are assumed not to conflict with any other effects and are used when the dynamic type of receiver methods cannot be known statically due to dynamically dispatched method invocation. Although open effects are somewhat analogous to surface effects, in the sense that disjointness information may be unknown until runtime, there is a subtle difference: while open effects are inductively defined and designed for optimistic parallelism, where concretized effects soundly approximate runtime effects, surface effects are syntactically defined in terms of unevaluated expressions and their concretization derives from runtime heap allocation. The correctness of surface effects is always statically known and the purpose of dynamic checking is to find further possibilities for parallelism. Moreover, surface effects can be used in those scenarios where conflict decision based on static effects would reject parallel programs. Since more precise effect specifications, yet statically verified, are provided directly by the programmer, surface effect analysis can override static effect analysis.

Irregular data structures such as trees, queues, and graphs motivate the use of run-time information because region-level analyses may merge accesses that are concrete-address disjoint. The Galois approach (Kulkarni et al., 2007) is a useful contrast because it uses run-time conflict detection in an optimistic execution model. Surface effects instead evaluate a summary before the parallel computation and therefore do not require the calculus to roll back a computation after a summary-level conflict is found. This is a semantic difference, not an established performance comparison.

The present work supplies neither a cost semantics nor an implementation study. Consequently, we make no claim that summary evaluation is constant in the size of the summary, that pre-checking has lower total overhead than access logging, or that useful parallelism is recovered frequently in practice. A future evaluation should measure at least summary-evaluation time, conflict-test time, false-positive rate, the frequency of ‘Top’, recovered parallelism relative to region-only static effects, and programmer annotation effort. These measurements are necessary to determine whether the formal precision mechanism translates into a practical advantage.

10 Conclusions

We have presented a region-polymorphic calculus in which a computation is paired with a statically verified, executable effect-summary term. Evaluating the summary before the computation can expose concrete region/location actions that a region-only static effect does not distinguish. The mechanized correctness theorem proves trace approximation, and the scheduler-soundness lemma makes the critical bridge explicit: disjoint and non-conflicting computed summaries of deterministic component traces yield a deterministic parallel trace. The trace semantics then establishes equivalent terminating replay heaps. The continuation-machine theorem stack adds finite-prefix safety: well-typed prefixes preserve typing, heap shape, trace typing, and not-stuckness, and the staged parallel-tuple check is connected either to a checked interleaving target or to the sequential fallback path.

These results are formal, not empirical performance results. The calculus does not include summary inference, a termination argument for summary evaluation, a cost semantics, an implementation evaluation, or a separate adequacy theorem for terminal continuation-machine runs. Establishing the annotation burden and the run-time trade-off between summary precision, pre-checking cost, and recovered parallelism is therefore left to future work.

References

Manuel Arenaz, Juan Touriño, and Ramón Doallo. An inspector-executor algorithm for irregular assignment parallelization. In *Parallel and Distributed Processing and Applications*, volume 3358 of *Lecture Notes in Computer Science*, pages 4–15. Springer, 2004. doi: 10.1007/978-3-540-30566-8_4.

Robert L. Bocchino, Jr., Vikram S. Adve, Danny Dig, Sarita V. Adve, Stephen Heumann, Rakesh Komuravelli, Jeffrey Overbey, Patrick Simmons, Hyojin Sung, and Mohsen Vakilian. A type and effect system for deterministic parallel Java. *SIGPLAN Not.*, 44(10):97–116, 2009.

Arthur Charguéraud. The locally nameless representation. *Journal of Automated Reasoning*, 49(3): 363–408, 2012.

Cormac Flanagan, Stephen N. Freund, and Aaron Tomb. Hybrid types, invariants, and refinements for imperative objects. In *International Workshop on Foundations and Developments of Object-Oriented Languages (FOOL)*, 2006.

Ming Kawaguchi, Patrick Rondon, Alexander Bakst, and Ranjit Jhala. Deterministic parallelism via liquid effects. *SIGPLAN Not.*, 47(6):45–54, 2012.

Milind Kulkarni, Keshav Pingali, Bruce Walter, Ganesh Ramanarayanan, Kavita Bala, and L. Paul Chew. Optimistic parallelism requires abstractions. *SIGPLAN Not.*, 42(6):211–222, 2007.

Yuheng Long, Mehdi Bagherzadeh, and Hridesh Rajan. Open effects: A hybrid type-and-effect system to tackle the open world assumption and its application to optimistic concurrency. Technical Report TR13-04a, Iowa State University, 2013.

John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '88)*, pages 47–57, New York, NY, USA, 1988. ACM.

Alexandros Tzannes, Stephen T. Heumann, Lamyaa Eloussi, Mohsen Vakilian, Vikram S. Adve, and Michael Han. Region and effect inference for safe parallelism. *Automated Software Engineering*, 26 (2):463–509, 2019. doi: 10.1007/s10515-019-00257-3.

A Deterministic Parallelism

Definition 1 (Heap equivalence)**.** *The mechanization represents a heap as a finite map from keys (r, l) to values. We write $H_1 \equiv H_2$ when the two finite maps are extensionally equal, equivalently when every key has the same lookup result in both heaps:*

$$H_1 \equiv H_2 \quad \iff \quad \forall k.\, H_1(k) = H_2(k).$$

This is equality of key–value maps, not an isomorphism modulo renaming of locations. The relation is reflexive, symmetric, and transitive.

Definition 2 (Deterministic trace)**.** *The predicate $\mathsf{Det}(\phi)$ is inductively generated by the empty trace, single dynamic actions, sequential composition of deterministic traces, and a parallel constructor. The latter requires deterministic component traces and both*

$$\neg\mathsf{ConflictTraces}(\phi_1, \phi_2) \qquad \textit{and} \qquad \mathsf{DisjointTraces}(\phi_1, \phi_2).$$

Thus absence of an explicitly represented read–write or write–write conflict is not the only premise: every pair of dynamic actions drawn from opposite branches must also satisfy dynamic-action disjointness.

Definition 3 (Trace replay)**.** *The relation $\Rightarrow$ executes one dynamic action of a trace against a heap. Allocation and write actions update the heap, reads require the recorded value to be present, and sequential/parallel trace nodes provide the evaluation contexts. We write $\Rightarrow^*$ for its reflexive transitive closure. A terminating replay has the form $(\phi, H) \Rightarrow^* (\mathsf{Nil}, H')$.*

Lemma 3 (Read-only trace properties)**.** *A read-only trace preserves the heap during replay. Two read-only component traces form a deterministic parallel trace. These results follow from the inductive definition of* ReadOnly *on traces, which contains only the empty trace, read actions, and sequential/-parallel compositions of read-only traces.*

Lemma 4 (Scheduler soundness)**.** *For $i \in \{1, 2\}$, let $\phi_i \sqsubseteq \Theta_i$ and $\mathsf{Det}(\phi_i)$. If*

$$\mathsf{Disjoint}(\Theta_1, \Theta_2) \quad \wedge \quad \neg(\Theta_1 \otimes \Theta_2),$$

then $\mathsf{Det}(\phi_1 \parallel \phi_2)$.

The proof uses the soundness relation $\sqsubseteq$ in both directions required by the parallel-trace constructor: a dynamic conflict would imply a computed conflict, and computed-action disjointness implies disjointness of every represented pair of dynamic actions.

Theorem 8 (Confluence of deterministic trace replay)**.** *Let $H_a \equiv H_b$. If a deterministic trace ϕ is replayed from the two equivalent heaps to intermediate configurations (ϕ_1, H_1) and (ϕ_2, H_2), then there exist a common trace ϕ_3 and heaps $H_3 \equiv H_4$ such that*

$$(\phi_1, H_1) \Rightarrow^* (\phi_3, H_3) \qquad \text{and} \qquad (\phi_2, H_2) \Rightarrow^* (\phi_3, H_4).$$

In particular, if both replays terminate at Nil*, their final heaps are equivalent.*

The result does not claim uniqueness of an intermediate trace independently of the stated confluence property.

Theorem 9 (Heap uniqueness for safe parallel summaries)**.** *Let $\phi_1 \sqsubseteq \Theta_1$ and $\phi_2 \sqsubseteq \Theta_2$, with deterministic component traces and*

$$\mathsf{Disjoint}(\Theta_1, \Theta_2) \quad \wedge \quad \neg(\Theta_1 \otimes \Theta_2).$$

Let $H_a \equiv H_b$. If

$$(\phi_1 \parallel \phi_2, H_a) \Rightarrow^* (\mathsf{Nil}, H_1) \quad \text{and} \quad (\phi_1 \parallel \phi_2, H_b) \Rightarrow^* (\mathsf{Nil}, H_2),$$

then $H_1 \equiv H_2$.

The proof applies scheduler soundness to establish determinism of the parallel trace and then uses the terminating replay theorem.